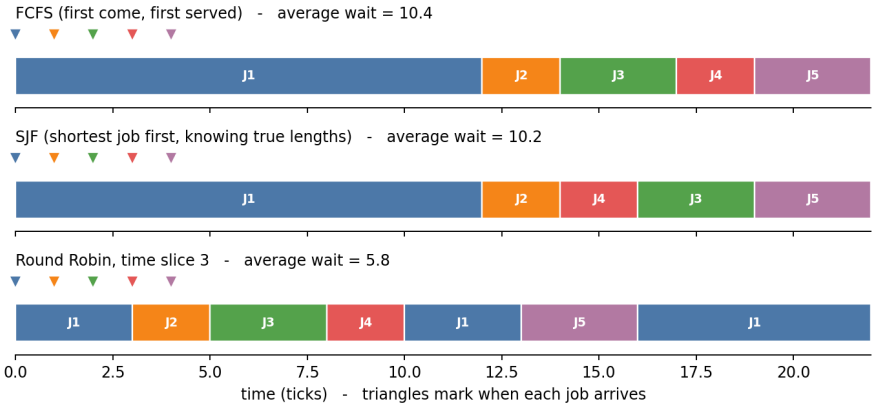
Adaptive CPU Scheduling: teammate briefing

The whole project in 4 sentences. A CPU can run only one program at a time, so the OS needs a rule for who goes next (FCFS, SJF or Round Robin). No single rule wins everywhere. We built a simulator that can **switch rule while running**, plus a small selector that **tests each rule on the recent past and switches only when another one clearly would have done better**. We tested it on made-up workloads and on two real supercomputer logs, and report honestly where it helps and where it does not.



Same five jobs, three rules. Round Robin wins here (5.8) but is the worst when jobs are long and similar (36.4 vs 20.2), so no rule is best everywhere.

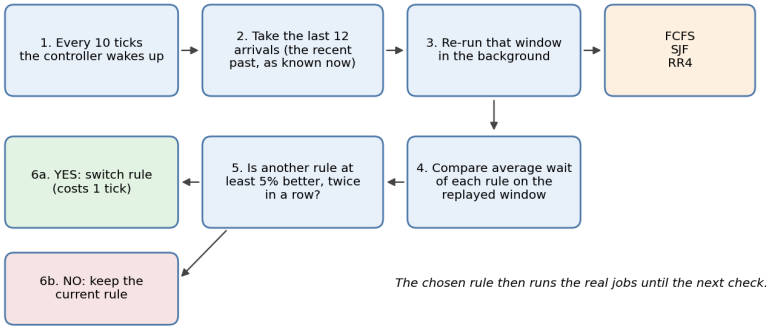
The claims we make (and exactly how to say them)

#	Claim	Evidence (numbers)	Say it like this
1	We built a correct, tested simulator that can change rule mid-run.	33 automated tests, incl. textbook values and 150 random cases.	“It reproduces the textbook answers, so we trust it.”
2	The selector is **never the worst** and limits worst-case damage.	Synthetic, 100 seeds: worst-case regret 30% vs 60% for plain SJF; mean regret 8.2% vs 11.8%.	“It is a safety net: we do not need to know the workload type in advance.”
3	It does **not** beat always-SJF when jobs are predictable.	Only 0.03–1.4 ticks worse on four easy workloads; clearly better (−10.5) on the ‘spiky’ one.	“If guesses are good, plain SJF is hard to beat.”
4	On real logs it is never worst, but not best.	SP2: ties SJF. Paragon: beats SJF (80% of slices), not Round Robin.	“Real jobs are heavy-tailed, so replaying 12 jobs is a noisy sample.”
5	**Better guesses of job length mattered more than switching.**	SP2: using each user’s requested time (for first jobs, and corrected by their learned habit) made SJF clearly better than a plain running average: about 17% with history, up to 40% on cold-start slices. The ‘average of the last two jobs’ rule alone gave no clear gain once history existed. With good guesses, always-SJF beats the selector (765 vs 954).	“Our biggest win was using what users tell us, corrected by their habits, not switching.”
6	Some ideas failed, and we report them.	Old if/else router (23.7% vs 11.8% regret), guess-reliability guard, tail-latency requirement.	“We tested them honestly; they did not pay off.”

What we do not claim

- A new scheduling algorithm. The idea is known as *portfolio scheduling*, and runtime prediction is a long-studied topic. Our contribution is a careful, reproducible small-scale evaluation.
- That it works in a real operating system (it is a simulator), or on every kind of workload (two real logs, one with requested times; parallel jobs treated as single-CPU bursts).

How the selector works, and a 2-minute way to explain it



Every 10 ticks: replay the last 12 arrivals under each rule, and switch only if another rule is at least 5% better twice in a row.

- **1. The problem:** show the picture on page 1. Different jobs need different rules.
- **2. The idea:** instead of guessing which rule suits the workload, *try them on recent history* and pick the winner.
- **3. The safeguards:** the 5% margin and “twice in a row” stop it flip-flopping on noise.
- **4. The honest finding:** it protects us from bad choices, but good guesses of job length help more than any switching.
- **5. The honesty:** three of our own ideas failed and we say so; the core idea already exists in the literature.

Six words your teammate will be asked about

Term	Plain meaning
Offered load	How much work arrives compared with what the CPU can do. 0.9 = nearly full, where choices matter; above 1 = the queue grows forever. Real logs were rescaled to 0.9.
Regret	How much worse than the best single rule for that run, in %. Low is good.
SJF*	SJF that cheats by knowing true job lengths. A ceiling for comparison, never a competitor.
p95 / max wait	The wait that 95% of jobs beat / the worst wait. They show the unlucky jobs that the average hides.
Estimator	How the scheduler guesses a job’s length when it arrives (running average, last two jobs, requested time, corrected requested time).
Dev vs report seeds	Settings were tuned on separate random workloads (seeds 1000+) so the reported results (seeds 0–99) are not tuned on.

If you want to extend it (best payoff first)

Idea	Effort	Why it is worth doing	Risk
More real logs that include requested times; rerun the estimator study	Low (1–2 days)	The estimator finding rests on one log (SP2). A second log makes it far more convincing.	Low. The result may differ, and that is still reportable.
Better estimators on the same logs: per-user median, simple machine learning, a second ‘ratio’ variant	Low–med.	The corrected estimator was our clearest win; a stronger estimator may widen it (perfect knowledge is still ~40% better).	Low. It may not beat ‘corrected’.
Add SRTF and MLFQ to the rules the selector can choose from	Medium	SJF is non-preemptive now; a richer portfolio gives adaptation more room to help.	Medium. It may still prefer plain SJF.
Learned selector: a small decision tree trained on which rule won, compared with replay	Medium	Tests whether learning beats replay, and is cheap at run time and still explainable.	Medium
Use the processor counts in the logs (parallel jobs, backfilling)	High	Removes our biggest simplification (single-CPU treatment of parallel jobs).	High effort
Prototype in Linux sched_ext or on real measured bursts	High	Connects the simulation to a real kernel.	High. It is a separate project.

My suggestion: do the first two (small, and they strengthen what you already claim; the Tsafrir predictor itself is now implemented and tested), then SRTF plus the decision-tree selector if there is time. Whatever you add, report the result even if it does not win.